

✓ AI Lab Experiments

Python, Anaconda/Jupyter, NumPy, Pandas, Matplotlib, Scikit-learn

✓ Experiment 1: Install Python, Anaconda/Jupyter Notebook, and AI Libraries

Aim: Install Python, Anaconda/Jupyter Notebook, and the required AI libraries (NumPy, Pandas, Matplotlib, Scikit-learn). Verify installation by importing each library and displaying its version.

Installation steps (run in Anaconda Prompt / terminal, not in this notebook):

```
# 1. Download and install Anaconda from https://www.anaconda.com/download
# 2. Anaconda includes Python, Jupyter Notebook, NumPy, Pandas, Matplotlib, Scikit-learn by default

# 3. To launch Jupyter Notebook:
jupyter notebook

# 4. If any library is missing, install it using pip or conda:
pip install numpy pandas matplotlib scikit-learn
# OR
conda install numpy pandas matplotlib scikit-learn
```

```
# Verify installation by importing each library and displaying its version
import sys
import numpy as np
import pandas as pd
import matplotlib
import sklearn

print("Python version:", sys.version)
print("NumPy version:", np.__version__)
print("Pandas version:", pd.__version__)
print("Matplotlib version:", matplotlib.__version__)
print("Scikit-learn version:", sklearn.__version__)
```

```
Python version: 3.12.3 (main, Mar 3 2026, 12:15:18) [GCC 13.3.0]
NumPy version: 2.4.4
Pandas version: 3.0.2
Matplotlib version: 3.10.8
Scikit-learn version: 1.8.0
```

✓ Experiment 2: Matrix Operations using NumPy

Aim: A company stores sales data in matrix form. Write a Python program using NumPy to perform matrix addition, subtraction, multiplication, transpose, and inverse of a matrix.

```
import numpy as np

# Sales data of two branches (rows = products, columns = months) stored as matrices
A = np.array([[10, 20, 30],
              [15, 25, 35],
              [5, 10, 15]])

B = np.array([[2, 4, 6],
              [1, 3, 5],
              [7, 8, 9]])

print("Matrix A (Branch 1 Sales):\n", A)
print("\nMatrix B (Branch 2 Sales):\n", B)

# Addition
print("\nAddition (A + B):\n", A + B)

# Subtraction
```

```

print("\nSubtraction (A - B):\n", A - B)

# Multiplication (matrix multiplication)
print("\nMultiplication (A x B):\n", np.dot(A, B))

# Transpose
print("\nTranspose of A:\n", A.T)

# Inverse (A must be square and non-singular)
try:
    A_float = A.astype(float)
    inverse_A = np.linalg.inv(A_float)
    print("\nInverse of A:\n", inverse_A)
except np.linalg.LinAlgError:
    print("\nMatrix A is singular and cannot be inverted.")

```

```

Matrix A (Branch 1 Sales):
[[10 20 30]
 [15 25 35]
 [ 5 10 15]]

```

```

Matrix B (Branch 2 Sales):
[[2 4 6]
 [1 3 5]
 [7 8 9]]

```

```

Addition (A + B):
[[12 24 36]
 [16 28 40]
 [12 18 24]]

```

```

Subtraction (A - B):
[[ 8 16 24]
 [14 22 30]
 [-2  2  6]]

```

```

Multiplication (A x B):
[[250 340 430]
 [300 415 530]
 [125 170 215]]

```

```

Transpose of A:
[[10 15  5]
 [20 25 10]
 [30 35 15]]

```

```

Matrix A is singular and cannot be inverted.

```

Experiment 3: Load, Clean and Analyze Dataset using Pandas

Aim: A student dataset contains missing values. Write a Python program using Pandas to load the dataset, clean missing values, and calculate statistical information such as average marks and highest score.

```

import pandas as pd
import numpy as np

# Creating a sample student dataset with missing values
data = {
    'Name': ['Amit', 'Bala', 'Chitra', 'Divya', 'Esha', 'Farhan'],
    'Marks': [78, np.nan, 85, 90, np.nan, 60]
}

df = pd.DataFrame(data)
print("Original Dataset:\n", df)

# Check for missing values
print("\nMissing values per column:\n", df.isnull().sum())

# Cleaning missing values by filling with the mean of the column
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
print("\nDataset after cleaning missing values:\n", df)

# Statistical analysis
average_marks = df['Marks'].mean()
highest_score = df['Marks'].max()
lowest_score = df['Marks'].min()
topper = df.loc[df['Marks'].idxmax(), 'Name']

```

```
print("\nAverage Marks:", round(average_marks, 2))
print("Highest Score:", highest_score)
print("Lowest Score:", lowest_score)
print("Topper:", topper)
```

Original Dataset:

	Name	Marks
0	Amit	78.0
1	Bala	NaN
2	Chitra	85.0
3	Divya	90.0
4	Esha	NaN
5	Farhan	60.0

Missing values per column:

Name	0
Marks	2

dtype: int64

Dataset after cleaning missing values:

	Name	Marks
0	Amit	78.00
1	Bala	78.25
2	Chitra	85.00
3	Divya	90.00
4	Esha	78.25
5	Farhan	60.00

Average Marks: 78.25

Highest Score: 90.0

Lowest Score: 60.0

Topper: Divya

✓ Experiment 4: Data Visualization using Matplotlib

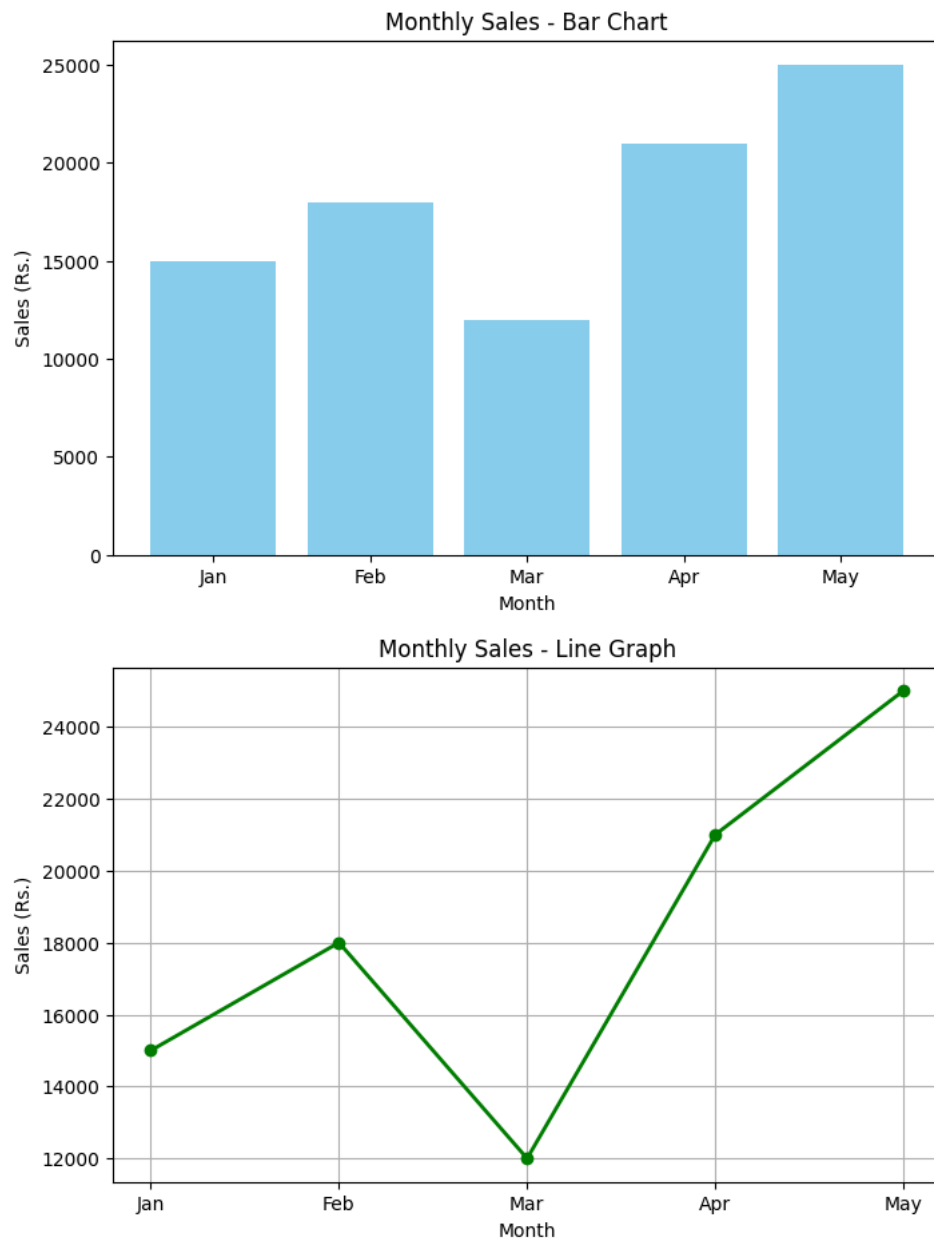
Aim: The sales of a shop for five months are given. Write a Python program using Matplotlib to visualize the monthly sales using a bar chart and line graph.

```
import matplotlib.pyplot as plt

months = ['Jan', 'Feb', 'Mar', 'Apr', 'May']
sales = [15000, 18000, 12000, 21000, 25000]

# Bar Chart
plt.figure(figsize=(8, 5))
plt.bar(months, sales, color='skyblue')
plt.title('Monthly Sales - Bar Chart')
plt.xlabel('Month')
plt.ylabel('Sales (Rs.)')
plt.show()

# Line Graph
plt.figure(figsize=(8, 5))
plt.plot(months, sales, marker='o', color='green', linewidth=2)
plt.title('Monthly Sales - Line Graph')
plt.xlabel('Month')
plt.ylabel('Sales (Rs.)')
plt.grid(True)
plt.show()
```



Experiment 5: Cosine Similarity between Two Text Vectors

Aim: Two documents are represented as numerical vectors. Write a Python program to calculate the cosine similarity between the two vectors using Scikit-learn and interpret the similarity score.

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# Two sample documents
doc1 = "Artificial intelligence is transforming the world of technology"
doc2 = "Technology is being transformed by artificial intelligence"

# Convert text documents into numerical vectors
vectorizer = CountVectorizer()
vectors = vectorizer.fit_transform([doc1, doc2])

print("Feature Names:\n", vectorizer.get_feature_names_out())
print("\nVector representation:\n", vectors.toarray())

# Calculate cosine similarity
similarity_score = cosine_similarity(vectors[0], vectors[1])
print("\nCosine Similarity Score:", similarity_score[0][0])
```

```
# Interpretation
score = similarity_score[0][0]
if score > 0.8:
    interpretation = "The two documents are highly similar."
elif score > 0.5:
    interpretation = "The two documents are moderately similar."
else:
    interpretation = "The two documents are not very similar."

print("Interpretation:", interpretation)
```

Feature Names:
['artificial' 'being' 'by' 'intelligence' 'is' 'of' 'technology' 'the'
 'transformed' 'transforming' 'world']

Vector representation:
[[1 0 0 1 1 1 1 1 0 1 1]
 [1 1 1 1 1 0 1 0 1 0 0]]

Cosine Similarity Score: 0.5345224838248487

Interpretation: The two documents are moderately similar.